

Windows network troubleshooting commands cheat sheet

Table of Contents

Piping the Output to Search	2
Ping an IP Address (by default 4 pings are sent)	2
Set packet size larger than default 32 bytes.	2
Send <i>n</i> number of pings	3
Send pings non-stop.	3
Network MTU maximum size path discovery (PMTU) testing with ping	3
Sweep Ping All Hosts in a Subnet (CIDR)	3
Traceroute to a Given IP Address	6
Connect via TCP to a Given Host/Port	6
Run TCP Port Scanner for a Host	7
Download File via HTTP/HTTPS	8
Show Routing Table of the Local Host	8
Resolve a Domain Name	9
Resolve MX Record Specifying Custom DNS Server	9
Show DNS SOA Record for a Domain	9
Show Local DNS Cached Resolved Names	9
Show Contents of hosts file	10
Show Windows Firewall Status for an Active Profile	10
Disable Windows Firewall	10
Enable Windows Firewall	10
Show Incoming/Inbound Rules of the Windows Firewall	11
Show Interfaces and their Status	11
Show Full Interface Parameters including MTU	11
Show All Network Connections with Ports and IPs	12
Show Only Ports in Listening State	12
Show ARP table	12
Show IPv6 Neighbors Table	12
List of Installed Patches and Hotfixes	13
Show Running Services	13
Show Current User	13
Show Time Zone Set and Current Time	14
Show Hosts Currently Visible and Active on local Host	14
Show Whether RDP is Enabled on Local Host	14
Show on What Port RDP is Enabled	14

Show Installed PowerShell Version	15
Related	15

Piping the Output to Search

In all PowerShell commands below, we can pipe the output of any command to `` | Out-String -Stream | Select-String "search_string"`` to search for a specific string/value. E.g.:

To search/show only connections to/from port 443:

```
Get-NetTCPConnection | Out-String -Stream | Select-String "443"
```

To search/show only route(s) to 10.0.0.33:

```
Get-NetRoute | Out-String -Stream | Select-String "10.0.0.33"
```

Ping an IP Address (by default 4 pings are sent)

cmd:

```
ping 8.8.8.8
```

PowerShell:

```
Test-Connection 8.8.8.8
```

Set packet size larger than default 32 bytes

NOTE | Windows will add 8 bytes of ICMP header to the custom size we specify

cmd:

```
ping -l 1200 8.8.8.8
```

PowerShell:

```
Test-Connection 8.8.8.8 -BufferSize 1200
```

Send n number of pings

cmd:

```
ping -n 1000 google.com
```

PowerShell:

```
Test-Connection google.com -Count 1000
```

Send pings non-stop

cmd:

```
ping 8.8.8.8 -t
```

- Older PowerShell up to version 7 (Windows 11 has version 5.1 by default):

PowerShell:

```
Test-Connection 8.8.8.8 -Count 10000
```

- Powershell 7 or newer:

```
Test-Connection 8.8.8.8 -Repeat
```

Network MTU maximum size path discovery (PMTU) testing with ping

Here I start pinging with 1400 bytes size, adding each time 20 bytes up to 2500 bytes:

cmd:

```
for /L %A in (1400,20,2500) do ping -f -l %A -n 2 8.8.8.8
```

Sweep Ping All Hosts in a Subnet (CIDR)

Here, I am pinging all hosts in the network 192.168.3.0/24, sending just 1 ICMP packet to each, and showing only results with Reply.

cmd:

```
for /L %i in (1,1,254) do @ping -n 1 192.168.3.%i | find "Reply"
```

NOTE

PowerShell below is to be saved in a file and run like: `powershell -ExecutionPolicy Bypass -File .\sweep.ps1 192.168.17.0/24`

PowerShell:

```
<#
.SYNOPSIS
    Ping-sweep a network given in CIDR notation and list only hosts that respond.
.PARAMETER Cidr
    Network in CIDR notation, e.g. 192.168.17.0/24
.PARAMETER TimeoutMs
    Reply timeout per host in milliseconds (default 1000).
.PARAMETER ResolveHostname
    Also perform a reverse-DNS lookup for each responding host (slower).
.EXAMPLE
    .\Ping-Sweep.ps1 192.168.17.0/24
.EXAMPLE
    .\Ping-Sweep.ps1 10.0.0.0/22 -TimeoutMs 500 -ResolveHostname
#>
[CmdletBinding()]
param(
    [Parameter(Mandatory = $true, Position = 0)]
    [string]$Cidr,

    [int]$TimeoutMs = 1000,

    [switch]$ResolveHostname
)

# ----- Parse the CIDR -----
if ($Cidr -notmatch '^(\d{1,3})(\.\d{1,3}){3}/\d{1,2}$') {
    throw "Invalid CIDR '$Cidr'. Expected format like 192.168.17.0/24"
}

$ipText, $prefixText = $Cidr -split '/'
$prefix = [int]$prefixText
if ($prefix -lt 0 -or $prefix -gt 32) {
    throw "Prefix length must be between 0 and 32 (got $prefix)."
}

$ipBytes = [System.Net.IPAddress]::Parse($ipText).GetAddressBytes()
[Array]::Reverse($ipBytes)
$ipInt = [BitConverter]::ToUInt32($ipBytes, 0)

if ($prefix -eq 0) {
    $mask = [uint32]0
} else {
    $mask = [uint32]::MaxValue -shl (32 - $prefix)
}
$invertedMask = $mask -bxor [uint32]::MaxValue
$network = $ipInt -band $mask
```

```

$broadcast = $network -bor $invertedMask

function ConvertTo-IPAddress([uint32]$Value) {
    $b = [BitConverter]::GetBytes($Value)
    [Array]::Reverse($b)
    return ([System.Net.IPAddress]$b).ToString()
}

# ----- Build the list of targets -----
$targets = @()
$hostBits = 32 - $prefix

if ($hostBits -ge 2) {
    for ($i = $network + 1; $i -lt $broadcast; $i++) {
        $targets += ConvertTo-IPAddress $i
    }
} else {
    for ($i = $network; $i -le $broadcast; $i++) {
        $targets += ConvertTo-IPAddress $i
    }
}

Write-Host "Sweeping $Cidr -> $($targets.Count) host(s)..." -ForegroundColor Cyan

# ----- Concurrent ping sweep -----
# One Ping object per host: each Ping allows only ONE async call at a time.
$jobs = @()
foreach ($ip in $targets) {
    $pinger = New-Object System.Net.NetworkInformation.Ping
    $jobs += [PSCustomObject]@{
        IP      = $ip
        Pinger  = $pinger
        Task    = $pinger.SendPingAsync($ip, $TimeoutMs)
    }
}

$alive = foreach ($job in $jobs) {
    try {
        $null = $job.Task.Wait($TimeoutMs + 500) # all tasks already run in parallel
        $reply = $job.Task.Result
        if ($reply.Status -eq [System.Net.NetworkInformation.IPStatus]::Success) {
            [PSCustomObject]@{
                IPAddress = $reply.Address.ToString()
                TimeMs    = $reply.RoundtripTime
            }
        }
    } catch {
        # host down / unreachable / timeout
    } finally {
        $job.Pinger.Dispose()
    }
}

```

```

}

# ----- Optional reverse DNS -----
if ($ResolveHostname -and $alive) {
    foreach ($host in $alive) {
        try {
            $host | Add-Member -NotePropertyName Hostname `
                -NotePropertyValue ([System.Net.Dns]::GetHostEntry($host.IPAddress)
                .HostName)
        } catch {
            $host | Add-Member -NotePropertyName Hostname -NotePropertyValue '(n/a)'
        }
    }
}

# ----- Output -----
if ($alive) {
    Write-Host "`n$($alive.Count) host(s) responded:`n" -ForegroundColor Green
    $alive | Sort-Object IPAddress | Format-Table -AutoSize
} else {
    Write-Host "`nNo hosts responded." -ForegroundColor Yellow
}

```

Traceroute to a Given IP Address

cmd:

```
tracert -d 8.8.8.8
```

PowerShell:

```
Test-NetConnection 8.8.8.8 -TraceRoute
```

Connect via TCP to a Given Host/Port

There is no more **telnet** installed by default, so no way to do it natively in cmd.exe, only PowerShell.

PowerShell:

```
Test-NetConnection 8.8.8.8 -Port 443
```

Answer should tell whether the connection succeeded or not:

```

ComputerName      : 8.8.8.8
RemoteAddress     : 8.8.8.8

```

```
RemotePort      : 443
InterfaceAlias   : ProtonVPN
SourceAddress    : 10.2.0.2
TcpTestSucceeded : True
```

Run TCP Port Scanner for a Host

`cmd` kind of version, as there is no `telnet` client we have to use PowerShell, but from CMD:

cmd:

```
for /L %p in (20,1,25) do @powershell -Command "if ((New-Object
Net.Sockets.TcpClient).ConnectAsync('192.168.3.13', %p).Wait(500)) { exit 0 } else {
exit 1 }" && echo Port %p is OPEN
```

- PowerShell (also async) version:

Here:

- Scan ports 20 to 25 inclusive
- Scan host 192.168.3.13
- Use timeout of 500 msec
- Scanning is asynchronous, not waiting for a given port to answer, so is fast

PowerShell:

```
20..25 | ForEach-Object {
    $client = New-Object System.Net.Sockets.TcpClient

    # Start an asynchronous connection attempt to the target port
    $async = $client.BeginConnect("192.168.3.13", $_, $null, $null)

    # Wait for 500 milliseconds for the connection to succeed
    $wait = $async.AsyncWaitHandle.WaitOne(500, $false)

    # If connected within the timeout, output the result and close the async hook
    if ($wait -and $client.Connected) {
        [PSCustomObject]@{
            Port    = $_
            Status  = "Open"
        }
        $client.EndConnect($async)
    }

    # Clean up the network resource
    $client.Close()
}
```

Output:

```
Port Status
-----
20 Open
21 Open
22 Open
25 Open
```

Download File via HTTP/HTTPS

WARNING

`certutil` is not really meant for file downloads, so while it works, there is 99% chance Microsoft Defender will block the download with notification on malware presence.

Here I explicitly allowed the download.

cmd:

```
certutil -urlcache -split -f "https://yurisk.info/assets/OSPF-fail-scenario.svg"
scenario.svg

*** Online ****
0000 ...
6b10
CertUtil: -URLCache command completed successfully.
```

PowerShell:

```
Invoke-WebRequest -Uri "https://github.com/yuriskinfo/cheat-sheets/blob/master/cheat-
sheets/fortigate-ssl-vpn-hardening-guide.pdf" -OutFile cheat-sheet.pdf
```

Show Routing Table of the Local Host

cmd:

```
route print
```

PowerShell:

```
Get-NetRoute
```

Resolve a Domain Name

cmd:

```
nslookup google.com
```

PowerShell:

```
Resolve-DnsName google.com
```

Resolve MX Record Specifying Custom DNS Server

cmd:

```
nslookup -type=MX yurisk.info 1.1.1.1
```

PowerShell:

```
Resolve-DnsName yurisk.info -Type MX -Server 1.1.1.1
```

Show DNS SOA Record for a Domain

cmd:

```
nslookup -type=SOA yurisk.info
```

PowerShell:

```
Resolve-DnsName yurisk.info -Type SOA
```

Show Local DNS Cached Resolved Names

cmd:

```
ipconfig /displaydns
```

PowerShell:

```
Get-DnsClientCache
```

Show Contents of hosts file

cmd:

```
type %SYSTEMROOT%\System32\drivers\etc\hosts
```

PowerShell:

```
Get-Content "$env:SystemRoot\System32\drivers\etc\hosts"
```

Show Windows Firewall Status for an Active Profile

cmd:

```
netsh advfirewall show currentprofile
```

PowerShell:

```
Get-NetConnectionProfile | ForEach-Object { Get-NetFirewallProfile -Name $_  
.NetworkCategory }
```

Disable Windows Firewall

NOTE

Must run as Administrator

cmd:

```
netsh advfirewall set allprofiles state off
```

PowerShell:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled False
```

Enable Windows Firewall

cmd:

```
netsh advfirewall set allprofiles state on
```

PowerShell:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True
```

Show Incoming/Inbound Rules of the Windows Firewall

cmd:

```
netsh advfirewall firewall show rule name=all dir=in
```

PowerShell:

```
Get-NetFirewallRule -Direction Inbound | `
Where-Object Enabled -eq "True" | `
Select-Object DisplayName, Action, Direction
```

Show Interfaces and their Status

cmd:

```
netsh interface show interface
```

PowerShell:

```
Get-NetAdapter | Select-Object Name, InterfaceDescription, Status, LinkSpeed
```

Show Full Interface Parameters including MTU

cmd:

```
ipconfig /all & netsh interface ipv4 show subinterfaces
```

PowerShell:

```
Get-NetIPInterface | Select-Object InterfaceAlias, AddressFamily, NlMtu,
ConnectionState, Dhcp
```

Show All Network Connections with Ports and IPs

cmd:

```
netstat -ano
```

PowerShell:

```
Get-NetTCPConnection | Select-Object LocalAddress, LocalPort, RemoteAddress,  
RemotePort, State
```

Show Only Ports in Listening State

cmd:

```
netstat -ano | findstr /R /C:"LISTENING"
```

PowerShell:

```
Get-NetTCPConnection -State Listen | `  
Select-Object LocalAddress, LocalPort, OwningProcess
```

Show ARP table

cmd:

```
arp -a
```

PowerShell:

```
Get-NetNeighbor -AddressFamily IPv4
```

Show IPv6 Neighbors Table

cmd:

```
netsh interface ipv6 show neighbors
```

PowerShell:

```
Get-NetNeighbor -AddressFamily IPv6
```

List of Installed Patches and Hotfixes

NOTE

Requires Admin privileges to run

cmd:

```
wmic qfe get HotFixID,Description,InstalledOn
```

-OR-

cmd:

```
dism /online /get-packages /format:table
```

PowerShell:

```
Get-HotFix
```

Show Running Services

cmd:

```
sc query
```

PowerShell:

```
Get-Service | Where-Object Status -eq 'Running'
```

Show Current User

cmd:

```
whoami
```

PowerShell:

```
[System.Security.Principal.WindowsIdentity]::GetCurrent().Name
```

Show Time Zone Set and Current Time

cmd:

```
tzutil /g & echo %DATE% %TIME%
```

PowerShell:

```
Get-TimeZone; Get-Date
```

Show Hosts Currently Visible and Active on local Host

PowerShell:

```
Get-NetNeighbor | Where-Object State -in ("Reachable", "Permanent") | `
Select-Object IPAddress, LinkLayerAddress, State
```

Show Whether RDP is Enabled on Local Host

cmd:

```
reg query "HKLM\System\CurrentControlSet\Control\Terminal Server" /v
fDenyTSConnections
```

NOTE | 0 means enabled, 1 means disabled.

PowerShell:

```
(Get-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server')
.fDenyTSConnections -eq 0
```

NOTE | True - enabled, False - disabled.

Show on What Port RDP is Enabled

cmd:

```
reg query "HKLM\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp"
/v PortNumber
```

NOTE | Answer is in hex, port 3389 will be 0xd3d.

PowerShell:

```
(Get-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal  
Server\WinStations\RDP-Tcp').PortNumber
```

Show Installed PowerShell Version

cmd:

```
powershell -Command "$PSVersionTable.PSVersion"
```

PowerShell:

```
$PSVersionTable.PSVersion
```

Related

- [Network MTU maximum size path discovery \(PMTU\) testing with ping](#)
- [Ping – setting don't fragment bit in Linux/Windows/FreeBSD/Solaris/Cisco/Juniper](#)

I also write cheat sheets/scripts/guides to help in a daily work at Github at <https://github.com/yuriskinfo> and <https://www.linkedin.com/in/yurislobodyanyuk/>